

**LAPORAN PRAKTIKUM  
PEMROGRAMAN MOBILE  
MODUL 5**



**Connect to the Internet**

**Oleh:**

**Rika Fauliana Rahmi   NIM. 2410817120017**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI 2026**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE**  
**MODUL 5**

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rika Fauliana Rahmi  
NIM : 2410817120017

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan  
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,  
M.Kom.  
NIP. 199710312025061009

## DAFTAR ISI

|                         |    |
|-------------------------|----|
| LEMBAR PENGESAHAN.....  | 2  |
| DAFTAR ISI .....        | 3  |
| DAFTAR GAMBAR.....      | 4  |
| DAFTAR TABEL .....      | 5  |
| SOAL 1 .....            | 6  |
| A. Source Code.....     | 6  |
| B. Output Program ..... | 27 |
| C. Pembahasan .....     | 28 |
| TAUTAN GIT .....        | 38 |

## **DAFTAR GAMBAR**

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 1. Screenshot Hasil Jawaban Soal 1 ..... | 27 |
| Gambar 2. Screenshot Hasil Jawaban Soal 1 ..... | 27 |
| Gambar 3. Screenshot Hasil Jawaban Soal 1 ..... | 28 |

## DAFTAR TABEL

|                                                          |    |
|----------------------------------------------------------|----|
| Tabel 1. Source Code MainActivity.kt Soal 1 .....        | 6  |
| Tabel 2. Source Code MyApplication.kt Soal 1 .....       | 9  |
| Tabel 3. Source Code MovieItem.kt Soal 1 .....           | 10 |
| Tabel 4. Source Code MovieViewModel.kt Soal 1 .....      | 12 |
| Tabel 5. Source Code AppScreen.kt Soal 1 .....           | 14 |
| Tabel 6. Source Code MovieDao.kt Soal 1 .....            | 19 |
| Tabel 7. Source Code MovieDatabase.kt Soal 1 .....       | 20 |
| Tabel 8. Source Code MovieEntity.kt Soal 1 .....         | 20 |
| Tabel 9. Source Code Movie.kt Soal 1 .....               | 21 |
| Tabel 10. Source Code TmdbResponse.kt Soal 1 .....       | 21 |
| Tabel 11. Source Code NetworkClient.kt Soal 1 .....      | 22 |
| Tabel 12. Source Code TmdbApiService.kt Soal 1 .....     | 23 |
| Tabel 13. Source Code MovieRepository.kt Soal 1 .....    | 23 |
| Tabel 14. Source Code ApiResult.kt Soal 1 .....          | 26 |
| Tabel 15. Source Code PreferencesManager.kt Soal 1 ..... | 26 |

## SOAL 1

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
  - a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
  - b. Gunakan KotlinX Serialization sebagai library JSON.
  - c. Gunakan library seperti Coil atau Glide untuk image loading.
  - d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
  - e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
  - f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
  - g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

### A. Source Code

#### Jetpack Compose

#### MainActivity.kt

*Tabel 1. Source Code MainActivity.kt Soal 1*

|   |                                                       |
|---|-------------------------------------------------------|
| 1 | package com.example.modul3compose                     |
| 2 |                                                       |
| 3 | import android.content.Intent                         |
| 4 | import android.net.Uri                                |
| 5 | import android.os.Bundle                              |
| 6 | import androidx.activity.ComponentActivity            |
| 7 | import androidx.activity.compose.setContent           |
| 8 | import androidx.compose.foundation.layout.fillMaxSize |

```

9 import androidx.compose.material3.MaterialTheme
10 import androidx.compose.material3.Surface
11 import androidx.compose.runtime.LaunchedEffect
12 import androidx.compose.runtime.collectAsState
13 import androidx.compose.runtime.getValue
14 import androidx.compose.ui.Modifier
15 import androidx.lifecycle.ViewModelProvider
16 import androidx.navigation.compose.NavHost
17 import androidx.navigation.compose.composable
18 import androidx.navigation.compose.rememberNavController
19 import
    com.example.modul3compose.ui.theme.Modul3composeTheme
20 import timber.log.Timber
21
22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25
26         val app = application as MyApplication
27         val viewModel = ViewModelProvider(
28             this,
29             MovieViewModel.Factory(app.movieRepository,
app.preferencesManager)
30         )[MovieViewModel::class.java]
31
32         setContent {
33             Modul3composeTheme {
34                 Surface(modifier
Modifier.fillMaxSize(), color
MaterialTheme.colorScheme.background) {
35
36                     val navController =
rememberNavController()
37                     val clickEvent by
viewModel.clickEvent.collectAsState()
38
39                     LaunchedEffect(clickEvent) {
40                         when (val event = clickEvent) {
41                             is
MovieViewModel.ClickEvent.DetailClick -> {
42                                 Timber.d("Berpindah ke
Detail → ${event.movie.title}")
43                                 navController.navigate("detail_screen/${event.movie.id}
")
44                                 viewModel.resetClickEvent()

```

```

45         }
46         is
MovieViewModel.ClickEvent.WebClick -> {
47             val url =
48             "https://www.themoviedb.org/movie/${event.movie.id}"
49
50             startActivity(Intent(Intent.ACTION_VIEW,
51             Uri.parse(url)))
52
53             viewModel.resetClickEvent()
54
55             }
56             null -> {}
57         }
58     }
59
60     NavHost(navController =
61     navController, startDestination = "list_screen") {
62         composable("list_screen") {
63             ListScreen(
64                 viewModel = viewModel,
65                 onDetailClick = { movie
66                 -> viewModel.onDetailClick(movie) },
67                 onWebClick = { movie ->
68                 viewModel.onWebClick(movie) },
69                 onNavigateToLanguage = {
70                 navController.navigate("language_screen") }
71             )
72         }
73
74         composable("detail_screen/{movieId}") { backStackEntry -
75         >
76             val id =
77             backStackEntry.arguments?.getString("movieId")?.toIntOr
78             Null()
79
80             val moviesState =
81             viewModel.moviesState.collectAsState()
82
83             val movies =
84             (moviesState.value
85             as?
86             com.example.modul3compose.util.ApiResult.Success)?.data
87
88             val selected = movies?.find
89             {
90                 it.id == id
91             } ?:
92             viewModel.selectedMovie.collectAsState().value
93
94             if (selected != null)
95             DetailScreen(movie = selected)
96         }
97
98         composable("language_screen") {

```



|    |                                       |
|----|---------------------------------------|
| 72 | viewModel) LanguageScreen(viewModel = |
| 73 | }                                     |
| 74 | }                                     |
| 75 | }                                     |
| 76 | }                                     |
| 77 | }                                     |
| 78 | }                                     |
| 79 | }                                     |

## MyApplication.kt

*Tabel 2. Source Code MyApplication.kt Soal 1*

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | package com.example.modul3compose                         |
| 2  |                                                           |
| 3  | import android.app.Application                            |
| 4  | import                                                    |
|    | com.example.modul3compose.data.local.MovieDatabase        |
| 5  | import                                                    |
|    | com.example.modul3compose.data.remote.NetworkClient       |
| 6  | import                                                    |
|    | com.example.modul3compose.data.repository.MovieRepository |
| 7  | import                                                    |
|    | com.example.modul3compose.util.PreferencesManager         |
| 8  |                                                           |
| 9  | class MyApplication : Application() {                     |
| 10 |                                                           |
| 11 | lateinit var movieRepository: MovieRepository             |
| 12 | lateinit var preferencesManager: PreferencesManager       |
| 13 |                                                           |
| 14 | override fun onCreate() {                                 |
| 15 | super.onCreate()                                          |
| 16 |                                                           |
| 17 | preferencesManager = PreferencesManager(this)             |
| 18 |                                                           |
| 19 | val movieDao =                                            |
|    | MovieDatabase.getInstance(this).movieDao()                |
| 20 |                                                           |
| 21 | movieRepository = MovieRepository(                        |
| 22 | apiService = NetworkClient.tmdbApiService,                |
| 23 | movieDao = movieDao,                                      |
| 24 | preferencesManager = preferencesManager                   |
| 25 | )                                                         |
| 26 | }                                                         |
| 27 | }                                                         |

## MovieItem.kt

Tabel 3. Source Code MovieItem.kt Soal 1

```
1 package com.example.modul3compose
2
3 import androidx.compose.foundation.layout.*
4 import
5     androidx.compose.foundation.shape.RoundedCornerShape
6 import androidx.compose.material3.*
7 import androidx.compose.runtime.Composable
8 import androidx.compose.ui.Modifier
9 import androidx.compose.ui.draw.clip
10 import androidx.compose.ui.graphics.Color
11 import androidx.compose.ui.layout.ContentScale
12 import androidx.compose.ui.res.stringResource
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.text.style.TextOverflow
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17 import coil.compose.AsyncImage
18 import com.example.modul3compose.data.model.Movie
19
20 @Composable
21 fun MovieItem(movie: Movie, onWebClick: () -> Unit,
22     onDetailClick: () -> Unit) {
23     Card(
24         colors = CardDefaults.cardColors(containerColor
25             = Color(0xFF2C2C2E)),
26         shape = RoundedCornerShape(16.dp),
27         modifier = Modifier.fillMaxWidth(),
28         elevation = CardDefaults.cardElevation(4.dp)
29     ) {
30         Row(modifier =
31             Modifier.padding(16.dp).fillMaxWidth()) {
32             AsyncImage(
33                 model = movie.posterUrl,
34                 contentDescription = null,
35                 modifier = Modifier
36                     .size(100.dp, 140.dp)
37                     .clip(RoundedCornerShape(12.dp)),
38                 contentScale = ContentScale.Crop
39             )
40             Column(modifier = Modifier.padding(start =
41                 16.dp).weight(1f)) {
42                 Row(modifier = Modifier.fillMaxWidth(),
43                     horizontalArrangement = Arrangement.SpaceBetween) {
44                     Text(
```

```

40         text = movie.title,
41         color = Color.White,
42         fontSize = 18.sp,
43         fontWeight = FontWeight.Bold,
44         modifier = Modifier.weight(1f),
45         maxLines = 1,
46         overflow =
TextOverflow.Ellipsis
47     )
48     Text(
49         text =
movie.releaseDate.take(4), // ambil tahun saja
50         color = Color.LightGray,
51         fontSize = 14.sp
52     )
53 }
54
55     Spacer(modifier =
Modifier.height(8.dp))
56
57     Row {
58         Text(
59             text =
stringResource(R.string.label_theme),
60             color = Color.LightGray,
61             fontSize = 13.sp,
62             fontWeight = FontWeight.Bold
63         )
64         Text(
65             text = " ★
${String.format("%.1f", movie.voteAverage)}",
66             color = Color.LightGray,
67             fontSize = 13.sp
68         )
69     }
70     Text(
71         text = movie.overview,
72         color = Color.Gray,
73         fontSize = 12.sp,
74         maxLines = 2,
75         overflow = TextOverflow.Ellipsis,
76         modifier = Modifier.padding(top =
4.dp)
77     )
78
79     Spacer(modifier =
Modifier.height(12.dp))

```

|    |                                                   |
|----|---------------------------------------------------|
| 80 |                                                   |
| 81 | Row(modifier = Modifier.fillMaxWidth(),           |
|    | horizontalArrangement = Arrangement.End) {        |
| 82 | TextButton(onClick = onWebClick) {                |
| 83 |                                                   |
|    | Text(stringResource(R.string.btn_web), color =    |
|    | Color(0xFF98A7F5))                                |
| 84 | }                                                 |
| 85 | Button(                                           |
| 86 | onClick = onDetailClick,                          |
| 87 | colors =                                          |
|    | ButtonDefaults.buttonColors(containerColor =      |
|    | Color(0xFF98A7F5))                                |
| 88 | ) {                                               |
| 89 |                                                   |
|    | Text(stringResource(R.string.btn_detail), color = |
|    | Color.Black)                                      |
| 90 | }                                                 |
| 91 | }                                                 |
| 92 | }                                                 |
| 93 | }                                                 |
| 94 | }                                                 |
| 95 | }                                                 |

## MovieViewModel.kt

Tabel 4. Source Code MovieViewModel.kt Soal 1

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | package com.example.modul3compose                         |
| 2  |                                                           |
| 3  | import androidx.lifecycle.ViewModel                       |
| 4  | import androidx.lifecycle.ViewModelProvider               |
| 5  | import androidx.lifecycle.viewModelScope                  |
| 6  | import com.example.modul3compose.data.model.Movie         |
| 7  | import                                                    |
|    | com.example.modul3compose.data.repository.MovieRepository |
| 8  | import com.example.modul3compose.util.ApiResult           |
| 9  | import com.example.modul3compose.util.PreferencesManager  |
| 10 | import kotlinx.coroutines.flow.MutableStateFlow           |
| 11 | import kotlinx.coroutines.flow.StateFlow                  |
| 12 | import kotlinx.coroutines.flow.asStateFlow                |
| 13 | import kotlinx.coroutines.launch                          |
| 14 | import timber.log.Timber                                  |
| 15 |                                                           |
| 16 | class MovieViewModel(                                     |
| 17 | private val repository: MovieRepository,                  |
| 18 | private val preferencesManager: PreferencesManager        |

```

19 ) : ViewModel() {
20
21     private val _moviesState =
MutableStateFlow<ApiResponse<List<Movie>>>>(ApiResponse.Loading)
22     val moviesState: StateFlow<ApiResponse<List<Movie>>>> =
_moviesState.asStateFlow()
23
24     private val _selectedMovie =
MutableStateFlow<Movie?>(null)
25     val selectedMovie: StateFlow<Movie?> =
_selectedMovie.asStateFlow()
26
27     private val _clickEvent =
MutableStateFlow<ClickEvent?>(null)
28     val clickEvent: StateFlow<ClickEvent?> =
_clickEvent.asStateFlow()
29
30     sealed class ClickEvent {
31         data class DetailClick(val movie: Movie) :
ClickEvent()
32         data class WebClick(val movie: Movie) :
ClickEvent()
33     }
34
35     init {
36         loadMovies()
37     }
38
39     fun loadMovies() {
40         viewModelScope.launch {
41             repository.getMovies("popular").collect {
result ->
42                 _moviesState.value = result
43             }
44         }
45     }
46
47     fun onDetailClick(movie: Movie) {
48         Timber.d("Tombol Detail ditekan →
${movie.title}")
49         _clickEvent.value =
ClickEvent.DetailClick(movie)
50     }
51
52     fun onWebClick(movie: Movie) {
53         Timber.d("Tombol Web ditekan → ${movie.title}")

```

|    |                                                                |
|----|----------------------------------------------------------------|
| 54 | _clickEvent.value = ClickEvent.WebClick(movie)                 |
| 55 | }                                                              |
| 56 |                                                                |
| 57 | fun resetClickEvent() {                                        |
| 58 | _clickEvent.value = null                                       |
| 59 | }                                                              |
| 60 |                                                                |
| 61 | fun setLanguage(language: String) {                            |
| 62 | preferencesManager.selectedLanguage = language                 |
| 63 | viewModelScope.launch {                                        |
| 64 | repository.clearCache("popular")                               |
| 65 | repository.clearCache("now_playing")                           |
| 66 | loadMovies()                                                   |
| 67 | }                                                              |
| 68 | }                                                              |
| 69 |                                                                |
| 70 | class Factory(                                                 |
| 71 | private val repository: MovieRepository,                       |
| 72 | private val preferencesManager: PreferencesManager             |
| 73 | ) : ViewModelProvider.Factory {                                |
| 74 | @Suppress("UNCHECKED_CAST")                                    |
| 75 | override fun <T : ViewModel> create(modelClass: Class<T>): T { |
| 76 | return MovieViewModel(repository, preferencesManager) as T     |
| 77 | }                                                              |
| 78 | }                                                              |
| 79 | }                                                              |

## AppScreen.kt

*Tabel 5. Source Code AppScreen.kt Soal 1*

|    |                                                        |
|----|--------------------------------------------------------|
| 1  | package com.example.modul3compose                      |
| 2  |                                                        |
| 3  | import android.app.Activity                            |
| 4  | import android.content.Context                         |
| 5  | import androidx.compose.foundation.clickable           |
| 6  | import androidx.compose.foundation.layout.*            |
| 7  | import androidx.compose.foundation.lazy.LazyColumn     |
| 8  | import androidx.compose.foundation.lazy.LazyRow        |
| 9  | import androidx.compose.foundation.lazy.items          |
| 10 | import                                                 |
|    | androidx.compose.foundation.shape.RoundedCornerShape   |
| 11 | import androidx.compose.material.icons.Icons           |
| 12 | import androidx.compose.material.icons.filled.Language |
| 13 | import androidx.compose.material3.*                    |

```

14 import androidx.compose.runtime.Composable
15 import androidx.compose.runtime.collectAsState
16 import androidx.compose.runtime.getValue
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.graphics.Color
21 import androidx.compose.ui.layout.ContentScale
22 import androidx.compose.ui.platform.LocalContext
23 import androidx.compose.ui.res.stringResource
24 import androidx.compose.ui.text.font.FontWeight
25 import androidx.compose.ui.unit.dp
26 import androidx.compose.ui.unit.sp
27 import coil.compose.AsyncImage
28 import com.example.modul3compose.data.model.Movie
29 import com.example.modul3compose.util.ApiResult
30 import java.util.Locale
31
32 @OptIn(ExperimentalMaterial3Api::class)
33 @Composable
34 fun ListScreen(
35     viewModel: MovieViewModel,
36     onDetailClick: (Movie) -> Unit,
37     onWebClick: (Movie) -> Unit,
38     onNavigateToLanguage: () -> Unit
39 ) {
40     val moviesState by
41     viewModel.moviesState.collectAsState()
42
43     Scaffold(
44         topBar = {
45             TopAppBar(
46                 title = {
47                     Text(stringResource(R.string.app_name),
48                         fontWeight =
49                         FontWeight.Bold) },
50                 actions = {
51                     IconButton(onClick =
52                     onNavigateToLanguage) {
53                         Icon(Icons.Default.Language,
54                             contentDescription = "Lang")
55                     }
56                 }
57             )
58         }
59     ) { padding ->
60         when (val state = moviesState) {
61             is ApiResult.Loading -> {

```

```

56         Box(modifier = Modifier.fillMaxSize(),
57         contentAlignment = Alignment.Center) {
58             CircularProgressIndicator()
59         }
60     }
61     is ApiResult.Error -> {
62         Box(modifier = Modifier.fillMaxSize(),
63         contentAlignment = Alignment.Center) {
64             Column(horizontalAlignment =
65             Alignment.CenterHorizontally) {
66                 Text("❌ ${state.message}",
67                 color = MaterialTheme.colorScheme.error)
68                 Spacer(modifier =
69                 Modifier.height(12.dp))
70                 Button(onClick = {
71                     viewModel.loadMovies() }) { Text("Coba Lagi") }
72             }
73         }
74     }
75     is ApiResult.Success -> {
76         LazyColumn(
77             modifier =
78             Modifier.padding(padding).fillMaxSize(),
79             contentPadding =
80             PaddingValues(16.dp),
81             verticalArrangement =
82             Arrangement.spacedBy(16.dp)
83         ) {
84             item {
85                 Text("Highlight Movies",
86                 fontWeight = FontWeight.Bold, fontSize = 18.sp)
87                 LazyRow(
88                     modifier =
89                     Modifier.padding(top = 8.dp),
90                     horizontalArrangement =
91                     Arrangement.spacedBy(12.dp)
92                 ) {
93                     items(state.data.take(5)) {
94                         movie ->
95                         AsyncImage(
96                             model =
97                             movie.posterUrl,
98                             contentDescription =
99                             movie.title,
100                             modifier = Modifier

```



```

88                                     .size(260.dp,
150.dp)
89
90                                     .clickable      {
onDetailClick(movie) },
91                                     contentScale    =
ContentScale.Crop
92                                     )
93                                     }
94                                     }
95                                     Spacer(modifier    =
Modifier.height(16.dp))
96                                     Text("All Movies", fontWeight =
FontWeight.Bold, fontSize = 18.sp)
97                                     }
98
99                                     items(state.data) { movie ->
100                                         MovieItem(
101                                             movie = movie,
102                                             onWebClick    =      {
onWebClick(movie) },
103                                             onDetailClick  =      {
onDetailClick(movie) }
104                                         )
105                                     }
106                                 }
107                            }
108                        }
109                    }
110                }
111
112 @Composable
113 fun DetailScreen(movie: Movie) {
114     Column(
115         modifier = Modifier
116             .fillMaxSize()
117             .padding(16.dp),
118         horizontalAlignment    =
Alignment.CenterHorizontally
119     ) {
120         AsyncImage(
121             model = movie.posterUrl,
122             contentDescription = null,
123             modifier = Modifier
124                 .size(200.dp)
125                 .clip(RoundedCornerShape(16.dp)),

```

```

126         contentScale = ContentScale.Crop
127     )
128     Spacer(modifier = Modifier.height(16.dp))
129     Text(text = movie.title, fontSize = 24.sp,
fontWeight = FontWeight.Bold)
130     Text(
131         text
132         =
133         "${stringResource(R.string.label_year)}
134         ${movie.releaseDate.take(4)}",
135         fontSize = 16.sp
136     )
137     Text(
138         text
139         =
140         "${stringResource(R.string.label_theme)}
141         ${String.format("%.1f", movie.voteAverage)}",
142         fontSize = 16.sp
143     )
144     Spacer(modifier = Modifier.height(16.dp))
145     Text(
146         text = movie.overview.ifEmpty { "Sinopsis
147         belum tersedia untuk bahasa ini. Coba ganti bahasa ke
148         English di menu pengaturan bahasa." },
149         fontSize = 14.sp,
150         color = Color.Gray
151     )
152 }
153
154 @Composable
155 fun LanguageScreen(viewModel: MovieViewModel) {
156     val context = LocalContext.current
157     Column(
158         modifier = Modifier.fillMaxSize(),
159         verticalArrangement = Arrangement.Center,
160         horizontalAlignment
161         =
162         Alignment.CenterHorizontally
163     ) {
164         Text(stringResource(R.string.title_language),
165             fontSize = 22.sp, fontWeight = FontWeight.Bold)
166         Spacer(modifier = Modifier.height(24.dp))
167         Button(onClick = {
168             viewModel.setLanguage("id-ID")
169             updateLocale(context, "in")
170         }) { Text("Bahasa Indonesia") }
171         Spacer(modifier = Modifier.height(8.dp))
172         Button(onClick = {
173             viewModel.setLanguage("en-US")

```

|     |                                                    |
|-----|----------------------------------------------------|
| 164 | updateLocale(context, "en")                        |
| 165 | }) { Text("English") }                             |
| 166 | }                                                  |
| 167 | }                                                  |
| 168 |                                                    |
| 169 | fun updateLocale(context: Context, lang: String) { |
| 170 | val locale = Locale(lang)                          |
| 171 | Locale.setDefault(locale)                          |
| 172 | val config = context.resources.configuration       |
| 173 | config.setLocale(locale)                           |
| 174 | context.resources.updateConfiguration(config,      |
|     | context.resources.displayMetrics)                  |
| 175 | (context as? Activity)?.recreate()                 |
| 176 | }                                                  |

## MovieDao.kt

*Tabel 6. Source Code MovieDao.kt Soal 1*

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.example.modul3compose.data.local         |
| 2  |                                                      |
| 3  | import androidx.room.Dao                             |
| 4  | import androidx.room.Insert                          |
| 5  | import androidx.room.OnConflictStrategy              |
| 6  | import androidx.room.Query                           |
| 7  | import kotlinx.coroutines.flow.Flow                  |
| 8  |                                                      |
| 9  | @Dao                                                 |
| 10 | interface MovieDao {                                 |
| 11 |                                                      |
| 12 | @Query("SELECT * FROM movies WHERE category =        |
|    | :category ORDER BY voteAverage DESC")                |
| 13 | fun getMoviesByCategory(category: String):           |
|    | Flow<List<MovieEntity>>                              |
| 14 |                                                      |
| 15 | @Insert(onConflict = OnConflictStrategy.REPLACE)     |
| 16 | suspend fun insertMovies(movies: List<MovieEntity>)  |
| 17 |                                                      |
| 18 | @Query("DELETE FROM movies WHERE category =          |
|    | :category")                                          |
| 19 | suspend fun deleteMoviesByCategory(category: String) |
| 20 |                                                      |
| 21 | @Query("SELECT cachedAt FROM movies WHERE category = |
|    | :category LIMIT 1")                                  |
| 22 | suspend fun getLastCacheTime(category: String):      |
|    | Long?                                                |
| 23 | }                                                    |

## MovieDatabase.kt

Tabel 7. Source Code MovieDatabase.kt Soal 1

```
1 package com.example.modul3compose.data.local
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7
8 @Database(entities = [MovieEntity::class], version = 1,
9 exportSchema = false)
10 abstract class MovieDatabase : RoomDatabase() {
11     abstract fun movieDao(): MovieDao
12
13     companion object {
14         @Volatile
15         private var INSTANCE: MovieDatabase? = null
16
17         fun getInstance(context: Context): MovieDatabase
18         {
19             return INSTANCE ?: synchronized(this) {
20                 Room.databaseBuilder(
21                     context.applicationContext,
22                     MovieDatabase::class.java,
23                     "movie_database"
24                 ).build().also { INSTANCE = it }
25             }
26         }
27     }
28 }
```

## MovieEntity.kt

Tabel 8. Source Code MovieEntity.kt Soal 1

```
1 package com.example.modul3compose.data.local
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "movies")
7 data class MovieEntity(
8     @PrimaryKey val id: Int,
9     val title: String,
10    val overview: String,
11    val posterPath: String?,
12 )
```

|    |                          |
|----|--------------------------|
| 12 | val releaseDate: String, |
| 13 | val voteAverage: Double, |
| 14 | val category: String,    |
| 15 | val cachedAt: Long       |
| 16 | )                        |

## Movie.kt

*Tabel 9. Source Code Movie.kt Soal 1*

|    |                                              |
|----|----------------------------------------------|
| 1  | package com.example.modul3compose.data.model |
| 2  |                                              |
| 3  | data class Movie(                            |
| 4  | val id: Int,                                 |
| 5  | val title: String,                           |
| 6  | val overview: String,                        |
| 7  | val posterPath: String?,                     |
| 8  | val releaseDate: String,                     |
| 9  | val voteAverage: Double,                     |
| 10 | val posterUrl: String                        |
| 11 | )                                            |

## TmdbResponse.kt

*Tabel 10. Source Code TmdbResponse.kt Soal 1*

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.example.modul3compose.data.model         |
| 2  |                                                      |
| 3  | import kotlinx.serialization.SerialName              |
| 4  | import kotlinx.serialization.Serializable            |
| 5  |                                                      |
| 6  | @Serializable                                        |
| 7  | data class TmdbResponse(                             |
| 8  | @SerialName("page") val page: Int = 0,               |
| 9  | @SerialName("results") val results: List<TmdbMovie>  |
|    | = emptyList(),                                       |
| 10 | @SerialName("total_pages") val totalPages: Int = 0,  |
| 11 | @SerialName("total_results") val totalResults: Int = |
|    | 0                                                    |
| 12 | )                                                    |
| 13 |                                                      |
| 14 | @Serializable                                        |
| 15 | data class TmdbMovie(                                |
| 16 | @SerialName("id") val id: Int,                       |
| 17 | @SerialName("title") val title: String = "",         |
| 18 | @SerialName("overview") val overview: String = "",   |
| 19 | @SerialName("poster_path") val posterPath: String? = |
|    | null,                                                |

|    |                                                         |
|----|---------------------------------------------------------|
| 20 | @SerializedName("release_date") val releaseDate: String |
| 21 | = "",                                                   |
| 22 | @SerializedName("vote_average") val voteAverage: Double |
| 23 | = 0.0,                                                  |
| 24 | @SerializedName("genre_ids") val genreIds: List<Int> =  |
|    | emptyList(),                                            |
|    | @SerializedName("popularity") val popularity: Double =  |
|    | 0.0                                                     |
|    | )                                                       |

## NetworkClient.kt

Tabel 11. Source Code NetworkClient.kt Soal 1

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | package com.example.modul3compose.data.remote             |
| 2  |                                                           |
| 3  | import com.example.modul3compose.BuildConfig              |
| 4  | import                                                    |
|    | com.jakewharton.retrofit2.converter.kotlinx.serialization |
|    | on.asConverterFactory                                     |
| 5  | import kotlinx.serialization.json.Json                    |
| 6  | import okhttp3.MediaType.Companion.toMediaType            |
| 7  | import okhttp3.OkHttpClient                               |
| 8  | import okhttp3.logging.HttpLoggingInterceptor             |
| 9  | import retrofit2.Retrofit                                 |
| 10 |                                                           |
| 11 | object NetworkClient {                                    |
| 12 |                                                           |
| 13 | private val json = Json {                                 |
| 14 | ignoreUnknownKeys = true                                  |
| 15 | coerceInputValues = true                                  |
| 16 | }                                                         |
| 17 |                                                           |
| 18 | private val loggingInterceptor =                          |
|    | HttpLoggingInterceptor().apply {                          |
| 19 | level = HttpLoggingInterceptor.Level.BODY                 |
| 20 | }                                                         |
| 21 |                                                           |
| 22 | private val okHttpClient = OkHttpClient.Builder()         |
| 23 | .addInterceptor(loggingInterceptor)                       |
| 24 | .build()                                                  |
| 25 |                                                           |
| 26 | val tmdbApiService: TmdbApiService by lazy {              |
| 27 | Retrofit.Builder()                                        |
| 28 | .baseUrl(BuildConfig.TMDB_BASE_URL)                       |
| 29 | .client(okHttpClient)                                     |

|    |                                                                                              |
|----|----------------------------------------------------------------------------------------------|
| 30 | <code>.addConverterFactory(json.asConverterFactory("application/json".toMediaType()))</code> |
| 31 | <code>.build()</code>                                                                        |
| 32 | <code>.create(TmdbApiService::class.java)</code>                                             |
| 33 | <code>}</code>                                                                               |
| 34 | <code>}</code>                                                                               |

## TmdbApiService.kt

Tabel 12. Source Code TmdbApiService.kt Soal 1

|    |                                                                       |
|----|-----------------------------------------------------------------------|
| 1  | <code>package com.example.modul3compose.data.remote</code>            |
| 2  |                                                                       |
| 3  | <code>import com.example.modul3compose.data.model.TmdbResponse</code> |
| 4  | <code>import retrofit2.http.GET</code>                                |
| 5  | <code>import retrofit2.http.Query</code>                              |
| 6  |                                                                       |
| 7  | <code>interface TmdbApiService {</code>                               |
| 8  |                                                                       |
| 9  | <code>    @GET("movie/popular")</code>                                |
| 10 | <code>    suspend fun getPopularMovies(</code>                        |
| 11 | <code>        @Query("api_key") apiKey: String,</code>                |
| 12 | <code>        @Query("language") language: String = "id-ID",</code>   |
| 13 | <code>        @Query("page") page: Int = 1</code>                     |
| 14 | <code>    ): TmdbResponse</code>                                      |
| 15 |                                                                       |
| 16 | <code>    @GET("movie/now_playing")</code>                            |
| 17 | <code>    suspend fun getNowPlayingMovies(</code>                     |
| 18 | <code>        @Query("api_key") apiKey: String,</code>                |
| 19 | <code>        @Query("language") language: String = "id-ID",</code>   |
| 20 | <code>        @Query("page") page: Int = 1</code>                     |
| 21 | <code>    ): TmdbResponse</code>                                      |
| 22 | <code>}</code>                                                        |

## MovieRepository.kt

Tabel 13. Source Code MovieRepository.kt Soal 1

|   |                                                                       |
|---|-----------------------------------------------------------------------|
| 1 | <code>package com.example.modul3compose.data.repository</code>        |
| 2 |                                                                       |
| 3 | <code>import com.example.modul3compose.BuildConfig</code>             |
| 4 | <code>import com.example.modul3compose.data.local.MovieDao</code>     |
| 5 | <code>import com.example.modul3compose.data.local.MovieEntity</code>  |
| 6 | <code>import com.example.modul3compose.data.model.Movie</code>        |
| 7 | <code>import</code>                                                   |
|   | <code>com.example.modul3compose.data.remote.TmdbApiService</code>     |
| 8 | <code>import com.example.modul3compose.util.ApiResult</code>          |
| 9 | <code>import com.example.modul3compose.util.PreferencesManager</code> |

```

10 import kotlinx.coroutines.flow.flow
11 import kotlinx.coroutines.flow.map
12
13 class MovieRepository(
14     private val apiService: TmdbApiService,
15     private val movieDao: MovieDao,
16     private val preferencesManager: PreferencesManager
17 ) {
18
19     fun      getMovies(category:      String):
kotlinx.coroutines.flow.Flow<ApiResult<List<Movie>>>      =
flow {
20         emit(ApiResult.Loading)
21
22         val      lastCacheTime      =
movieDao.getLastCacheTime(category)
23         val isCacheExpired = lastCacheTime == null ||
24             (System.currentTimeMillis()      -
lastCacheTime) > PreferencesManager.CACHE_DURATION_MS
25
26         if (!isCacheExpired) {
27             movieDao.getMoviesByCategory(category)
28                 .map      {      entities      ->
ApiResult.Success(entities.map { it.toMovie() }) }
29                 .collect { emit(it) }
30             return@flow
31         }
32
33         try {
34             val      language      =
preferencesManager.selectedLanguage
35             val response = when (category) {
36                 "now_playing"      ->
apiService.getNowPlayingMovies(
37                 apiKey = BuildConfig.TMDB_API_KEY,
38                 language = language
39             )
40             else -> apiService.getPopularMovies(
41                 apiKey = BuildConfig.TMDB_API_KEY,
42                 language = language
43             )
44         }
45
46         val      entities      =      response.results.map      {
tmdbMovie ->
47             MovieEntity(
48                 id = tmdbMovie.id,

```



```

49         title = tmdbMovie.title,
50         overview = tmdbMovie.overview,
51         posterPath = tmdbMovie.posterPath,
52         releaseDate = tmdbMovie.releaseDate,
53         voteAverage = tmdbMovie.voteAverage,
54         category = category,
55         cachedAt =
System.currentTimeMillis()
56     )
57 }
58 movieDao.deleteMoviesByCategory(category)
59 movieDao.insertMovies(entities)
60
61     movieDao.getMoviesByCategory(category)
62         .map { dbEntities ->
ApiResponse.Success(dbEntities.map { it.toMovie() }) }
63         .collect { emit(it) }
64
65     } catch (e: Exception) {
66         val cachedMovies =
movieDao.getLastCacheTime(category)
67         if (cachedMovies != null) {
68             movieDao.getMoviesByCategory(category)
69                 .map { entities ->
ApiResponse.Success(entities.map { it.toMovie() }) }
70                 .collect { emit(it) }
71         } else {
72             emit(ApiResult.Error(e.message ?:
"Terjadi kesalahan"))
73         }
74     }
75 }
76
77 suspend fun clearCache(category: String) {
78     movieDao.deleteMoviesByCategory(category)
79 }
80 }
81
82 fun MovieEntity.toMovie(): Movie {
83     return Movie(
84         id = id,
85         title = title,
86         overview = overview,
87         posterPath = posterPath,
88         releaseDate = releaseDate,
89         voteAverage = voteAverage,
90         posterUrl = if (posterPath != null)

```

|    |                                                   |
|----|---------------------------------------------------|
| 91 | "\${BuildConfig.TMDB_IMAGE_BASE_URL}\$posterPath" |
| 92 | else ""                                           |
| 93 | )                                                 |
| 94 | }                                                 |

## ApiResult.kt

Tabel 14. Source Code ApiResult.kt Soal 1

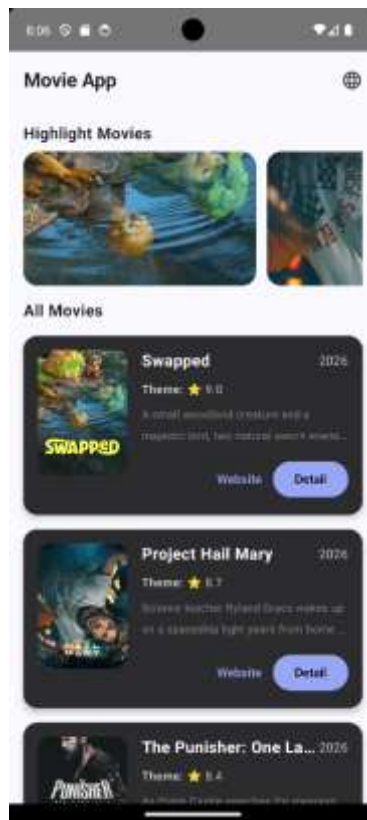
|   |                                                      |
|---|------------------------------------------------------|
| 1 | package com.example.modul3compose.util               |
| 2 |                                                      |
| 3 | sealed class ApiResult<out T> {                      |
| 4 | data class Success<T>(val data: T) : ApiResult<T>()  |
| 5 | data class Error(val message: String, val code: Int? |
| 6 | = null) : ApiResult<Nothing>()                       |
| 7 | object Loading : ApiResult<Nothing>()                |
| 8 | }                                                    |

## PreferencesManager.kt

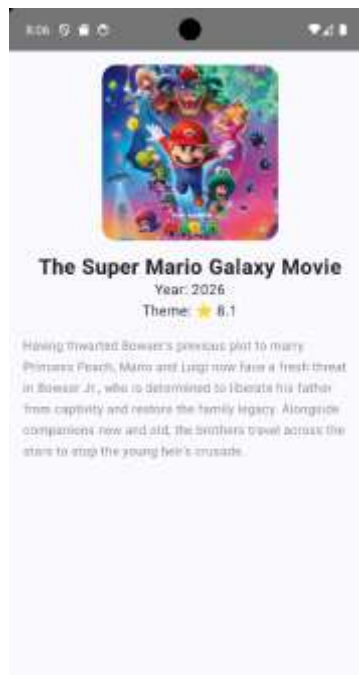
Tabel 15. Source Code PreferencesManager.kt Soal 1

|    |                                                     |
|----|-----------------------------------------------------|
| 1  | package com.example.modul3compose.util              |
| 2  |                                                     |
| 3  | import android.content.Context                      |
| 4  | import android.content.SharedPreferences            |
| 5  |                                                     |
| 6  | class PreferencesManager(context: Context) {        |
| 7  |                                                     |
| 8  | private val prefs: SharedPreferences =              |
| 9  | context.getSharedPreferences("movie_prefs",         |
| 10 | Context.MODE_PRIVATE)                               |
| 11 | companion object {                                  |
| 12 | private const val KEY_LANGUAGE =                    |
| 13 | "selected_language"                                 |
| 14 | const val CACHE_DURATION_MS = 60 * 60 * 1000L       |
| 15 | }                                                   |
| 16 | var selectedLanguage: String                        |
| 17 | get() = prefs.getString(KEY_LANGUAGE, "id-ID")      |
| 18 | ?: "id-ID"                                          |
| 19 | set(value)                                          |
| 20 | prefs.edit().putString(KEY_LANGUAGE, value).apply() |

## B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1



Gambar 2. Screenshot Hasil Jawaban Soal 1



*Gambar 3. Screenshot Hasil Jawaban Soal 1*

### **C. Pembahasan**

#### **MainActivity.kt**

Pada file MainActivity.kt, alur utama aplikasi dimulai dari fungsi 'onCreate()'. Setelah memanggil 'super.onCreate()', program menginisialisasi 'ViewModelProvider' untuk mengelola data. Antarmuka Jetpack Compose dibangun menggunakan fungsi 'setContent', dengan komponen 'Surface' yang diatur memenuhi layar lewat fungsi 'Modifier.fillMaxSize()' di bawah tema 'Modul3composeTheme'.

Navigasi antar layar dikendalikan oleh fungsi 'rememberNavController()', sementara perubahan data dipantau menggunakan fungsi 'collectAsState()'. Untuk menangani efek samping secara aktif, digunakan fungsi 'LaunchedEffect()'. Di dalamnya, jika terjadi klik detail film, aplikasi mencetak log dengan 'Timber.d', berpindah halaman lewat 'navController.navigate()', dan menyegarkan status dengan 'viewModel.resetClickEvent()'.

Jika klik mengarah ke web, browser luar dibuka menggunakan fungsi 'startActivity()' sebelum memanggil kembali 'viewModel.resetClickEvent()'.

Terakhir, komponen 'NavHost' mengatur rute navigasi yang masing-masing didefinisikan oleh fungsi 'composable()'. Halaman utama memanggil fungsi 'ListScreen' beserta fungsi interaksinya seperti 'viewModel.onDetailClick()' dan 'viewModel.onWebClick()'. Halaman detail akan mengumpulkan data state lewat 'collectAsState()' untuk menampilkan fungsi 'DetailScreen' berdasarkan ID yang sesuai. Sedangkan rute terakhir akan langsung memanggil fungsi 'LanguageScreen'.

### **MyApplication.kt**

Pada file MyApplication.kt, class ini bertindak sebagai basis global aplikasi yang mengatur inisialisasi dependensi sejak awal. Proses ini berjalan di dalam fungsi 'onCreate()', yang diawali dengan memanggil fungsi 'super.onCreate()' untuk menjalankan siklus hidup bawaan Android. Langkah pertama di dalamnya adalah menyiapkan manajemen konfigurasi lokal dengan memanggil konstruktor 'PreferencesManager()'. Untuk mengelola database lokal, program memanggil fungsi 'MovieDatabase.getInstance()' dan mengambil objek akses data melalui fungsi 'movieDao()'. Instance DAO dan pengatur preferensi tersebut, bersama dengan layanan API dari 'NetworkClient.tmdbApiService', kemudian dioper ke dalam konstruktor 'MovieRepository()' untuk menginisialisasi properti repositori utama. Pola ini memastikan seluruh penyediaan data siap diakses oleh komponen aplikasi lainnya semenjak aplikasi pertama kali diluncurkan.

### **MovieItem.kt**

Komponen utama yang didefinisikan adalah fungsi composable 'MovieItem()' yang menerima tiga parameter yaitu objek 'movie' bertipe 'Movie', serta dua parameter fungsi 'onWebClick' dan 'onDetailClick' yang akan dipanggil ketika tombol ditekan. Di dalam fungsi ini, komponen 'Card' digunakan sebagai wadah utama dengan warna latar belakang gelap, sudut membulat menggunakan 'RoundedCornerShape(16.dp)', dan bayangan menggunakan 'CardDefaults.cardElevation(4.dp)'.

Di dalam 'Card', komponen 'Row' digunakan untuk menyusun konten secara horizontal. Pada sisi kiri, komponen 'AsyncImage' dari library Coil memuat gambar poster

film secara asinkron dari URL 'movie.posterUrl' dengan properti 'ContentScale.Crop' agar gambar memenuhi area yang tersedia. Pada sisi kanan, komponen 'Column' menyusun informasi film secara vertikal mulai dari judul dan tahun rilis yang diambil menggunakan 'movie.releaseDate.take(4)', rating film yang diformat menggunakan 'String.format("%.1f", movie.voteAverage)', hingga sinopsis 'movie.overview' yang dibatasi dua baris menggunakan 'maxLines = 2' dan 'TextOverflow.Ellipsis'. Bagian bawah 'Column' berisi dua tombol yang disusun dengan 'Arrangement.End'. Tombol 'TextButton' memanggil 'onWebClick' untuk membuka halaman web film, sedangkan 'Button' memanggil 'onDetailClick' untuk berpindah ke halaman detail. Label kedua tombol diambil dari string resource menggunakan 'stringResource' agar mendukung multibahasa.

### **MovieViewModel.kt**

Pada file 'MovieViewModel.kt', kelas 'MovieViewModel' didefinisikan dengan mewarisi kelas 'ViewModel' dan menerima dua parameter yaitu 'repository' bertipe 'MovieRepository' dan 'preferencesManager' bertipe 'PreferencesManager'. Kelas ini bertugas mengelola state dan logika bisnis aplikasi.

Di dalam class ini, terdapat tiga 'MutableStateFlow' yang digunakan untuk menyimpan state. '\_moviesState' menyimpan state daftar film dengan nilai awal 'ApiResult.Loading', '\_selectedMovie' menyimpan film yang sedang dipilih, dan '\_clickEvent' menyimpan event klik yang terjadi. Ketiganya diekspos sebagai 'StateFlow' yang bersifat read-only menggunakan 'asStateFlow()' agar UI hanya bisa membaca tanpa mengubah langsung. Sealed class 'ClickEvent' didefinisikan untuk merepresentasikan dua jenis aksi yaitu 'DetailClick' untuk membuka halaman detail dan 'WebClick' untuk membuka halaman web film. Saat aplikasi pertama dijalankan, blok 'init' langsung memanggil fungsi 'loadMovies()' untuk memuat data film.

Fungsi 'loadMovies()' menjalankan coroutine menggunakan 'viewModelScope.launch' dan memanggil 'repository.getMovies("popular")' yang mengembalikan Flow, lalu setiap hasil yang masuk dari Flow diassign ke '\_moviesState.value'. Fungsi 'onDetailClick()' dan 'onWebClick()' masing-masing mencatat log menggunakan 'Timber.d()' dan mengisi '\_clickEvent' dengan event yang sesuai, sedangkan 'resetClickEvent()' mengosongkan kembali '\_clickEvent' menjadi null setelah

event ditangani. Fungsi 'setLanguage()' menyimpan preferensi bahasa baru ke 'preferencesManager.selectedLanguage', kemudian menghapus cache lama menggunakan 'repository.clearCache()' untuk kategori 'popular' dan 'now\_playing', lalu memanggil 'loadMovies()' agar data dimuat ulang dari API dengan bahasa yang baru. Terakhir, inner class 'Factory' mengimplementasikan 'ViewModelProvider.Factory' untuk menginisialisasi 'MovieViewModel' beserta dependensinya saat ViewModel dibuat.

## **AppScreen.kt**

Pada file 'AppScreens.kt', terdapat tiga fungsi composable utama:

Fungsi 'ListScreen()' menerima parameter 'viewModel', 'onDetailClick', 'onWebClick', dan 'onNavigateToLanguage'. Di dalam fungsi ini, state film diobservasi menggunakan 'viewModel.moviesState.collectAsState()'. Komponen 'Scaffold' digunakan sebagai kerangka halaman dengan 'TopAppBar' yang menampilkan nama aplikasi dan tombol ikon bahasa yang memanggil 'onNavigateToLanguage' saat ditekan. Konten utama menangani tiga kondisi state menggunakan blok 'when': jika 'ApiResponse.Loading' maka ditampilkan 'CircularProgressIndicator', jika 'ApiResponse.Error' maka ditampilkan pesan error beserta tombol "Coba Lagi" yang memanggil 'viewModel.loadMovies()', dan jika 'ApiResponse.Success' maka data film ditampilkan menggunakan 'LazyColumn'. Di dalam 'LazyColumn', bagian atas menampilkan 'LazyRow' berisi lima poster film highlight menggunakan 'state.data.take(5)' yang bisa diklik untuk navigasi ke detail, diikuti daftar lengkap film menggunakan komponen 'MovieItem'.

Fungsi 'DetailScreen()' menerima parameter 'movie' dan menampilkan detail film secara vertikal menggunakan 'Column'. Poster film ditampilkan menggunakan 'AsyncImage', diikuti judul, tahun rilis dari 'movie.releaseDate.take(4)', rating, dan sinopsis. Jika sinopsis kosong, fungsi 'isEmpty' menampilkan pesan pengganti yang menyarankan pengguna untuk mengganti bahasa ke English.

Fungsi 'LanguageScreen()' menampilkan dua tombol pilihan bahasa. Ketika tombol ditekan, fungsi 'viewModel.setLanguage()' dipanggil untuk memperbarui preferensi bahasa di ViewModel, kemudian fungsi 'updateLocale()' dipanggil untuk mengubah konfigurasi locale aplikasi dan memanggil 'recreate()' agar tampilan aplikasi diperbarui sesuai bahasa yang dipilih.

## MovieDao.kt

Pada file 'MovieDao.kt', interface 'MovieDao' didefinisikan dengan anotasi '@Dao' yang menandakan bahwa interface ini merupakan Data Access Object untuk mengakses database Room. Di dalam interface ini terdapat empat fungsi. Fungsi 'getMoviesByCategory()' menggunakan anotasi '@Query' untuk mengambil semua data film dari tabel 'movies' berdasarkan kategori tertentu, diurutkan dari rating tertinggi menggunakan 'ORDER BY voteAverage DESC'. Fungsi ini mengembalikan 'Flow<List<MovieEntity>>' agar perubahan data di database dapat diobservasi secara reaktif oleh UI.

Fungsi 'insertMovies()' menggunakan anotasi '@Insert' dengan parameter 'onConflict = OnConflictStrategy.REPLACE', artinya jika data dengan id yang sama sudah ada di database maka data lama akan diganti dengan data baru. Fungsi 'deleteMoviesByCategory()' menggunakan anotasi '@Query' untuk menghapus semua data film berdasarkan kategori tertentu, yang dipanggil sebelum menyimpan data baru agar cache lama tidak menumpuk. Fungsi 'getLastCacheTime()' mengambil nilai 'cachedAt' dari satu baris data menggunakan 'LIMIT 1' untuk mengecek kapan terakhir kali data kategori tersebut disimpan, yang digunakan untuk menentukan apakah cache sudah kadaluarsa atau belum.

## MovieDatabase.kt

Pada file 'MovieDatabase.kt', class abstrak 'MovieDatabase' didefinisikan dengan mewarisi 'RoomDatabase' dan dianotasi '@Database' yang mendaftarkan 'MovieEntity' sebagai tabel, menetapkan versi database sebagai 1, dan menonaktifkan ekspor skema dengan 'exportSchema = false'. Di dalam kelas ini terdapat fungsi abstrak 'movieDao()' yang mengembalikan instance 'MovieDao' untuk mengakses operasi database.

Pada blok 'companion object', variabel 'INSTANCE' dideklarasikan dengan anotasi '@Volatile' agar perubahan nilainya langsung terlihat oleh semua thread. Fungsi 'getInstance()' mengimplementasikan pola Singleton, jika 'INSTANCE' masih null maka database dibuat menggunakan 'Room.databaseBuilder()' dengan nama 'movie\_database', lalu disimpan ke 'INSTANCE' menggunakan 'also'. Blok 'synchronized(this)' memastikan hanya satu thread yang bisa membuat instance database pada satu waktu, sehingga tidak terjadi pembuatan database ganda.



## **MovieEntity.kt**

Pada file 'MovieEntity.kt', data class 'MovieEntity' didefinisikan dengan anotasi '@Entity(tableName = "movies")' yang menandakan bahwa kelas ini merepresentasikan sebuah tabel bernama 'movies' di dalam database Room. Setiap properti di dalam kelas ini menjadi kolom pada tabel tersebut.

Properti 'id' dianotasi dengan '@PrimaryKey' sebagai kunci utama tabel yang bersumber dari id film di TMDB API. Properti lainnya meliputi 'title' untuk judul film, 'overview' untuk sinopsis, 'posterPath' yang bersifat nullable untuk path gambar poster, 'releaseDate' untuk tanggal rilis, 'voteAverage' bertipe 'Double' untuk menyimpan rating film, 'category' untuk membedakan apakah film termasuk kategori 'popular' atau 'now\_playing', serta 'cachedAt' bertipe 'Long' untuk menyimpan timestamp kapan data terakhir kali disimpan ke cache yang digunakan dalam pengecekan ekspirasi cache.

## **Movie.kt**

Pada file 'Movie.kt', data class 'Movie' didefinisikan sebagai model utama yang digunakan oleh UI untuk menampilkan data film. Berbeda dengan 'MovieEntity' yang merepresentasikan struktur tabel database, kelas 'Movie' merepresentasikan data yang sudah siap dikonsumsi oleh tampilan aplikasi. Properti yang dimiliki meliputi 'id' sebagai identitas unik film, 'title' untuk judul, 'overview' untuk sinopsis, 'posterPath' yang bersifat nullable untuk menyimpan path relatif poster dari TMDB, 'releaseDate' untuk tanggal rilis, 'voteAverage' bertipe 'Double' untuk rating film, serta 'posterUrl' yang menyimpan URL lengkap gambar poster yang sudah digabungkan dengan base URL TMDB sehingga bisa langsung digunakan oleh 'AsyncImage' untuk memuat gambar.

## **TmdbResponse.kt**

Pada file 'TmdbResponse.kt', terdapat dua data class yang digunakan untuk memetakan respons JSON dari TMDB API menggunakan library KotlinX Serialization.

Data class 'TmdbResponse' dianotasi '@Serializable' dan merepresentasikan struktur respons utama dari API. Setiap properti dianotasi '@SerializedName' untuk mencocokkan nama field JSON dari API, seperti 'page' untuk nomor halaman, 'results' berisi daftar film bertipe 'List<TmdbMovie>', 'total\_pages' untuk total halaman, dan 'total\_results' untuk total

keseluruhan film. Semua properti memiliki nilai default sehingga tidak akan error jika field tersebut tidak ada di respons JSON.

Data class 'TmdbMovie' juga dianotasi '@Serializable' dan merepresentasikan struktur data satu film di dalam array 'results'. Properti yang dipetakan meliputi 'id', 'title', 'overview', 'poster\_path' yang bersifat nullable, 'release\_date', 'vote\_average', 'genre\_ids', dan 'popularity'. Data class ini kemudian akan dikonversi menjadi 'MovieEntity' sebelum disimpan ke Room, atau menjadi 'Movie' untuk ditampilkan di UI.

### **NetworkClient.kt**

Pada file 'NetworkClient.kt', object singleton 'NetworkClient' didefinisikan untuk mengatur konfigurasi jaringan dan menyediakan instance 'TmdbApiService' yang siap digunakan. Di dalam object ini, variabel 'json' dikonfigurasi menggunakan 'Json' dari KotlinX Serialization dengan dua properti yaitu 'ignoreUnknownKeys = true' agar field JSON yang tidak ada di model diabaikan dan tidak menyebabkan error, serta 'coerceInputValues = true' agar nilai null dari API dipaksa menggunakan nilai default yang sudah didefinisikan di data class. Selanjutnya, 'loggingInterceptor' dikonfigurasi menggunakan 'HttpLoggingInterceptor' dengan level 'BODY' untuk mencatat seluruh isi request dan response jaringan di logcat, yang berguna saat proses debugging. Interceptor ini kemudian dipasang ke 'OkHttpClient' melalui 'addInterceptor()'.

Properti 'tmdbApiService' dibuat menggunakan 'by lazy' agar instance Retrofit hanya dibuat satu kali ketika pertama kali diakses. Di dalamnya, 'Retrofit.Builder()' dikonfigurasi dengan 'baseUrl' yang diambil dari 'BuildConfig.TMDB\_BASE\_URL', menggunakan 'okHttpClient' yang sudah dikonfigurasi sebelumnya, dan menambahkan converter 'json.asConverterFactory()' agar Retrofit dapat mengonversi JSON secara otomatis menggunakan KotlinX Serialization. Terakhir, fungsi 'create()' dipanggil untuk menghasilkan implementasi dari interface 'TmdbApiService'.

### **TmdbApiService.kt**

Pada file 'TmdbApiService.kt', interface 'TmdbApiService' didefinisikan sebagai kontrak endpoint TMDB API yang akan diimplementasikan secara otomatis oleh Retrofit. Interface ini memiliki dua fungsi suspend. Fungsi 'getPopularMovies()' dianotasi

'@GET("movie/popular")' untuk mengakses endpoint daftar film populer, sedangkan fungsi 'getNowPlayingMovies()' dianotasi '@GET("movie/now\_playing")' untuk mengakses endpoint film yang sedang tayang. Kedua fungsi bersifat 'suspend' agar dapat dijalankan di dalam coroutine tanpa memblokir thread utama, dan keduanya mengembalikan 'TmdbResponse' yang akan otomatis dikonversi dari JSON oleh KotlinX Serialization.

Setiap fungsi menerima tiga parameter yang dianotasi '@Query' untuk dikirim sebagai query parameter pada URL request. Parameter 'apiKey' untuk autentikasi ke TMDB API, 'language' dengan nilai default 'id-ID' untuk menentukan bahasa respons yang diinginkan, dan 'page' dengan nilai default '1' untuk menentukan halaman data yang diambil.

### **MovieRepository.kt**

Pada file 'MovieRepository.kt', kelas 'MovieRepository' didefinisikan sebagai penghubung antara sumber data remote (TMDB API) dan lokal (Room Database), menerima tiga parameter yaitu 'apiService', 'movieDao', dan 'preferencesManager'. Fungsi utama 'getMovies()' mengembalikan 'Flow<ApiResult<List<Movie>>>' dan langsung memancarkan 'ApiResult.Loading' di awal menggunakan 'emit()'. Selanjutnya, fungsi 'movieDao.getLastCacheTime()' dipanggil untuk mengecek kapan terakhir kali data disimpan. Jika cache masih valid (belum melewati batas waktu di 'PreferencesManager.CACHE\_DURATION\_MS'), data langsung diambil dari Room menggunakan 'movieDao.getMoviesByCategory()' tanpa memanggil API, lalu dipetakan ke 'ApiResult.Success' menggunakan fungsi 'map' dan di-'collect' untuk di-'emit' ke UI.

Jika cache sudah kadaluarsa, aplikasi masuk ke blok 'try' dan memanggil API sesuai kategori menggunakan blok 'when'. Hasil respons API kemudian dikonversi menjadi list 'MovieEntity' dengan menambahkan 'cachedAt' berisi waktu saat ini menggunakan 'System.currentTimeMillis()'. Data lama dihapus menggunakan 'movieDao.deleteMoviesByCategory()' sebelum data baru disimpan menggunakan 'movieDao.insertMovies()', lalu data terbaru dari Room di-'emit' ke UI. Jika terjadi error pada blok 'catch', aplikasi mengecek apakah cache lama masih ada — jika ada maka data lama tetap ditampilkan, jika tidak ada maka 'ApiResult.Error' di-'emit' ke UI.

Fungsi 'clearCache()' merupakan fungsi suspend yang memanggil 'movieDao.deleteMoviesByCategory()' untuk menghapus cache berdasarkan kategori,

dipanggil saat pengguna mengganti bahasa agar data dimuat ulang dari API. Terakhir, extension function 'toMovie()' bertugas mengkonversi 'MovieEntity' menjadi 'Movie' dengan menggabungkan 'TMDB\_IMAGE\_BASE\_URL' dengan 'posterPath' untuk menghasilkan URL lengkap gambar poster.

### **ApiResponse.kt**

Pada file 'ApiResponse.kt', sealed class 'ApiResponse' didefinisikan sebagai generic response wrapper yang merepresentasikan tiga kemungkinan state dari sebuah operasi API. Sealed class ini memiliki tiga subclass. Subclass 'Success' menyimpan data hasil response yang berhasil melalui properti 'data' bertipe generik 'T', sehingga bisa digunakan untuk berbagai tipe data. Subclass 'Error' menyimpan pesan kesalahan melalui properti 'message' bertipe String dan properti opsional 'code' bertipe 'Int?' untuk menyimpan kode HTTP error jika ada. Subclass 'Loading' didefinisikan sebagai 'object' karena tidak membutuhkan data apapun, hanya merepresentasikan kondisi saat data sedang dimuat. Ketiga subclass ini digunakan di seluruh aplikasi untuk menangani state API secara konsisten melalui blok 'when' di UI.

### **PreferencesManager.kt**

Pada file 'PreferencesManager.kt', kelas 'PreferencesManager' didefinisikan untuk mengelola penyimpanan data ringan menggunakan SharedPreferences. Kelas ini menerima parameter 'context' dan menginisialisasi 'prefs' menggunakan 'context.getSharedPreferences("movie\_prefs", Context.MODE\_PRIVATE)' agar file preferensi hanya bisa diakses oleh aplikasi sendiri. Di dalam 'companion object', terdapat dua konstanta yaitu 'KEY\_LANGUAGE' sebagai kunci penyimpanan preferensi bahasa, dan 'CACHE\_DURATION\_MS' yang menetapkan durasi cache selama satu jam dalam satuan milidetik yang digunakan oleh 'MovieRepository' untuk mengecek apakah cache sudah kadaluarsa.

Properti 'selectedLanguage' didefinisikan menggunakan custom getter dan setter. Getter memanggil 'prefs.getString()' dengan nilai default 'id-ID' jika belum pernah disimpan sebelumnya, sedangkan setter memanggil 'prefs.edit().putString().apply()' untuk menyimpan

nilai bahasa baru secara asinkron ke SharedPreferences sehingga preferensi bahasa tetap tersimpan walau pengguna keluar dari aplikasi.

#### Caching Strategy: Cache-First with Time-based Expiry

Aplikasi menggunakan strategi Cache-First with Time-based Expiry, yaitu aplikasi mengutamakan pengambilan data dari penyimpanan lokal (Room Database) terlebih dahulu sebelum mengakses API. Cache dianggap kadaluarsa setelah 1 jam, dan baru akan fetch ulang dari TMDB API ketika waktu tersebut terlampaui.

Alasan:

1. Data film tidak berubah setiap saat, sehingga tidak perlu selalu hit API setiap kali aplikasi dibuka
2. Performa lebih cepat karena membaca dari database lokal lebih cepat dibanding request jaringan

## TAUTAN GIT

<https://github.com/rikafaulianarahmi/MODUL-MOBILE/tree/main/MODUL%205/Compose>